

Prompt:

8/17/2026, 9:58:53 PM

lanjut Tahap N

Response:

8/17/2026, 9:58:54 PM

Remembering

Recalled staged trading formulations and connected QUBO with risk sizing

Worked for 5s

Tahap N — Dynamic QUBO Construction & Position Sizing

Sekarang kita masuk ke tahap yang menurut saya **menjadi jantung matematis sistem trading kita**.

Pada Tahap M kita sudah memiliki:

$$\hat{\mu}_t$$

sebagai expected return,

dan

$$\hat{\Sigma}_t$$

sebagai covariance/risk structure.

Tetapi keduanya masih berada pada level **continuous portfolio theory**.

Trading nyata membutuhkan keputusan diskret:

aset mana yang dibeli, berapa besar posisinya, long atau short, pada harga berapa, dengan stop-loss berapa, dan apakah kombinasi seluruh posisi tersebut masih memenuhi risk budget?

Maka Tahap N bertugas melakukan transformasi:

$(\hat{\mu}_t, \hat{\Sigma}_t) \rightarrow$ Trading Optimization $\rightarrow$ QUBO $\rightarrow$ Ising Hamiltonian

dan pada akhirnya menghasilkan:

$$x_t^*$$

yang dapat diberikan kepada execution engine.

N.1. Masalah sebenarnya yang ingin kita selesaikan

Misalkan terdapat N aset.

Untuk setiap aset i , kita mengetahui:

$$P_i$$

harga saat ini,

$$\mu_i$$

expected return,

$$\sigma_i^2$$

variance,

dan:

$$\Sigma_{ij}$$

covariance.

Kita ingin menentukan:

$$w_i.$$

Secara klasik:

$$\max_{\mathbf{w}} [\hat{\boldsymbol{\mu}}^T \mathbf{w} - \gamma \mathbf{w}^T \Sigma \mathbf{w}].$$

Tetapi ini belum cukup untuk trading kita.

N.2. Mengapa Markowitz biasa belum cukup?

Markowitz menganggap:

$$w_i \in \mathbb{R}.$$

Padahal sistem kita memiliki keputusan seperti:

BUY

atau:

DO NOTHING

atau:

SHORT.

Selain itu kita memiliki:

- minimum order size,
- maximum position,
- \$1 risk budget,
- stop-loss,
- take-profit,
- transaction cost,
- slippage,
- liquidity,
- maximum number of simultaneous positions.

Jadi problem sebenarnya adalah:

Mixed discrete-constrained portfolio optimization.

N.3. Kita mulai dari satu trade

Sebelum membuat portfolio, mari kita pahami satu posisi.

Misalkan kita ingin long asset i .

Harga:

$$P_i.$$

Stop-loss:

$$SL_i.$$

Definisikan stop distance:

$$d_i = P_i - SL_i$$

untuk long.

Jika kita membeli:

$$q_i$$

unit, maka approximate maximum loss:

$$R_i = q_i d_i.$$

Kita ingin:

$$R_i \leq R_{\max}.$$

Dengan risk budget:

$$R_{\max} = \$1.$$

Maka:

$$q_i d_i \leq 1.$$

Sehingga:

$$q_i \leq \frac{1}{d_i}.$$

Ini adalah dasar position sizing kita.

N.4. Position sizing bukan berdasarkan modal

Ini perbedaan penting.

Sistem kita tidak berkata:

Saya memiliki \$1000, maka saya membeli \$100 BTC.

Sebaliknya:

Berapa ukuran posisi yang menghasilkan maksimal \$1 loss jika SL tersentuh?

Secara sederhana:

$$q_i = \frac{R_{\max}}{d_i}.$$

Dengan:

$$R_{\max} = 1.$$

N.5. Contoh

Misalkan:

$$P_i = 100$$

dan:

$$SL_i = 98.$$

Maka:

$$d_i = 2.$$

Dengan:

$$R_{\max} = 1,$$

kita memperoleh:

$$q_i = \frac{1}{2} = 0.5.$$

Notional:

$$N_i = q_i P_i = 50.$$

Jadi posisi \$50 memiliki stop-loss sekitar \$1.

N.6. Tetapi kita tidak ingin q_i selalu kontinu

Quantum optimization paling natural ketika keputusan kita diskret.

Maka kita melakukan **discretization**.

Misalkan kita memiliki level position:

$$q_{i,1}, q_{i,2}, \dots, q_{i,L}.$$

Contohnya:

$$L = 3$$

dengan:

$$q_{i,1} = 0.25q_i^{\max},$$

$$q_{i,2} = 0.50q_i^{\max},$$

$$q_{i,3} = 1.00q_i^{\max}.$$

N.7. Binary encoding

Definisikan binary variables:

$$x_{i,k} \in \{0, 1\}.$$

Dengan:

$$x_{i,k} = 1$$



berarti level k dipilih.

Position:

$$q_i = \sum_{k=1}^L q_{i,k} x_{i,k}.$$

N.8. Tetapi kita harus mencegah memilih dua level sekaligus

Misalnya:

$$x_{i,1} = 1$$

dan:

$$x_{i,3} = 1.$$

Maka kita mendapatkan:

$$q_i = q_{i,1} + q_{i,3},$$

yang tidak kita inginkan jika level tersebut mutually exclusive.

Constraint:

$$\sum_k x_{i,k} \leq 1.$$

N.9. Mengubah constraint menjadi penalty

Karena QUBO membutuhkan objective tanpa constraint eksplisit, kita masukkan penalty:

$$P_i \left(\sum_k x_{i,k} - 1 \right)^2.$$

Tetapi jika zero position diperbolehkan, kita sebenarnya membutuhkan:

$$\sum_k x_{i,k} \leq 1.$$

Untuk binary variable, bentuk penalty yang lebih tepat dapat menggunakan:

$$P_i \sum_{k < l} x_{i,k} x_{i,l}.$$

Jika dua level dipilih sekaligus, penalty muncul.

N.10. Sekarang kita masukkan expected return

Untuk position:

$$q_i,$$

expected PnL:

$$E[\Pi_i] \approx P_i q_i \hat{\mu}_i.$$

Definisikan:

$$a_{i,k} = P_i q_{i,k} \tilde{\mu}_i.$$

Dengan:

$$\tilde{\mu}_i = \hat{\mu}_i - C_i.$$

Maka contribution:

$$\Pi_i^{expected} = \sum_k a_{i,k} x_{i,k}.$$

N.11. Mengapa kita menggunakan net alpha?

Karena expected return mentah:

$$\hat{\mu}_i$$

belum mempertimbangkan:

$$fee + spread + slippage + impact.$$



Maka:

$$\tilde{\mu}_i = \hat{\mu}_i - C_i.$$

Kalau:

$$\tilde{\mu}_i < 0,$$

trade tersebut secara expected value sudah buruk bahkan sebelum risk diperhitungkan.

N.12. Sekarang masukkan portfolio risk

Position vector dalam notional:

$$w_i = P_i q_i.$$

Maka:

$$w^T \Sigma w$$

adalah portfolio variance.

Karena:

$$w_i = P_i \sum_k q_{i,k} x_{i,k},$$

kita memperoleh:

$$Var(R_p) = \sum_i \sum_j w_i w_j \Sigma_{ij}.$$

N.13. Ekspansi binary

Substitusi:

$$w_i = P_i \sum_k q_{i,k} x_{i,k}.$$

Maka:

$$Var(R_p) = \sum_{i,j} P_i P_j \Sigma_{ij} \left(\sum_k q_{i,k} x_{i,k} \right) \left(\sum_l q_{j,l} x_{j,l} \right).$$

Sehingga:

$$Var(R_p) = \sum_{i,j} \sum_{k,l} P_i P_j q_{i,k} q_{j,l} \Sigma_{ij} x_{i,k} x_{j,l}.$$

Inilah yang membuat masalah kita menjadi QUBO.

N.14. Objective utama

Kita ingin:

$$\max [Expected\ PnL - Risk\ Penalty - Trading\ Cost - Constraint\ Penalty].$$

Secara matematis:

$$\max_x [\tilde{\mu}^T w - \gamma w^T \Sigma w - P(x)].$$

N.15. Karena QUBO biasanya ditulis sebagai minimization

Kita ubah tanda:

$$\min_x [-\tilde{\mu}^T w + \gamma w^T \Sigma w + P(x)].$$

Definisikan:

$$E(x) = -\tilde{\mu}^T w + \gamma w^T \Sigma w + P(x).$$

Maka:

$$x^* = \arg \min_x E(x).$$

N.16. Dari sini QUBO muncul secara alami

Karena:

$$x_i^2 = x_i$$

untuk:

$$x_i \in \{0, 1\},$$

semua objective dapat ditulis:

$$E(x) = \sum_a Q_{aa}x_a + \sum_{a < b} Q_{ab}x_ax_b + C.$$

atau:

$$E(x) = x^T Q x + C.$$

N.17. Index a sekarang berarti apa?

Kita gabungkan:

$$a = (i, k).$$

Artinya satu binary variable mewakili:

memilih asset i pada position level k .

Misalnya:

$$x_{BTC,1}$$

berarti:

pilih BTC dengan position level 1.

N.18. Struktur QUBO kita

Secara konseptual:

$$Q = Q^{return} + Q^{risk} + Q^{cost} + Q^{constraint}.$$

Ini sangat berguna karena kita dapat menganalisis setiap komponen secara terpisah.

N.19. Return term

Untuk variable $x_{i,k}$:

$$Q_{(i,k)}^{return} = -P_i q_{i,k} \tilde{\mu}_i.$$

Semakin besar expected profit:

$$\tilde{\mu}_i,$$

semakin negatif diagonal QUBO.

Karena optimizer ingin:

$$E \rightarrow minimum.$$

N.20. Risk diagonal

Variance asset:

$$\Sigma_{ii}.$$

Contribution:

$$\gamma(P_i q_{i,k})^2 \Sigma_{ii}.$$

Sehingga:

$$Q_{aa}^{risk} = \gamma(P_i q_{i,k})^2 \Sigma_{ii}.$$

N.21. Risk interaction

Untuk dua posisi:

$$(i, k)$$

dan:

$$(j, l),$$

interaction:

$$Q_{ab}^{risk} \propto \gamma P_i P_j q_{i,k} q_{j,l} \Sigma_{ij}.$$

Interpretasinya sangat menarik.

Jika:

$$\Sigma_{ij} > 0,$$

memilih kedua posisi bersama menghasilkan penalty.

Jika:

$$\Sigma_{ij} < 0,$$

kombinasi tersebut dapat menurunkan portfolio risk.

N.22. Ini sebenarnya inti "quantitative trading"

Misalnya:

$$A$$

memiliki expected return tinggi.

Tetapi:

$$A$$

sangat berkorelasi dengan:

B .

Optimizer dapat berkata:

A bagus.

Tetapi:

A + B terlalu redundant.

Sebaliknya:

$A + C$

mungkin lebih baik karena:

$\text{Corr}(A, C) < 0$.

Jadi optimizer tidak hanya mencari:

best asset.

Ia mencari:

best combination.

N.23. Maximum number of positions

Kita mungkin tidak ingin membuka 20 trade sekaligus.

Definisikan:

$$y_i = \sum_k x_{i,k}.$$

Karena:

$$y_i \in \{0, 1\}$$

jika level position dibuat mutually exclusive.

Constraint:

$$\sum_i y_i \leq K_{max}.$$

N.24. Penalty cardinality

Kita dapat menggunakan:

$$P_K \left(\sum_i y_i - K_{target} \right)^2.$$

Tetapi ini memaksa tepat K_{target} .

Jika yang kita inginkan adalah **maksimal** K_{max} , penalty dapat dibuat dengan auxiliary variables atau formulasi inequality-to-equality.

Untuk V1, saya menyarankan:

$$\sum_i y_i \leq K_{max}$$

ditangani melalui slack variable.

N.25. Slack variable

Misalnya:

$$\sum_i y_i + s = K_{max},$$

dengan:

$$s \geq 0.$$

Kemudian:

$$s$$

direpresentasikan dalam binary encoding:

$$s = \sum_m 2^m z_m.$$

Constraint menjadi:

$$P_K \left(\sum_i y_i + \sum_m 2^m z_m - K_{max} \right)^2.$$

Ini kemudian masuk QUBO.

N.26. Risk budget portfolio

Sekarang masuk constraint yang lebih penting.

Kita ingin total worst-case stop-loss risk:

$$\sum_i R_i \leq R_{portfolio}.$$

Dengan:

$$R_i = q_i d_i.$$

Maka:

$$\sum_i \sum_k q_{i,k} d_i x_{i,k} \leq R_{portfolio}.$$

N.27. Contoh

Jika:

$$R_{portfolio} = \$5,$$

dan:

$$R_i = \$1$$

untuk setiap posisi, maka maksimal lima posisi secara kasar dapat dibuka.

Tetapi optimizer juga melihat covariance.

Jadi mungkin ia memilih:

3

posisi saja jika ketiganya sangat correlated.

N.28. Ini penting

Risk budget dan variance risk **bukan hal yang sama**.

Risk budget:

$$\sum_i R_i.$$

Variance:

$$w^T \Sigma w.$$

Kita membutuhkan keduanya.

Risk budget menjawab:

Berapa banyak uang yang bisa hilang jika SL tersentuh?

Variance menjawab:

Seberapa besar ketidakpastian portfolio secara statistik?

N.29. Position sizing yang lebih realistis

Sebelumnya:

$$q_i = \frac{1}{d_i}.$$

Tetapi volatility juga harus dipertimbangkan.

Misalkan:

$$\sigma_i$$

besar.

Maka position yang secara SL-risk aman belum tentu secara portfolio volatility aman.

Jadi kita memiliki dua filter:

Stop Risk

dan:

Statistical Risk.

N.30. Volatility-adjusted risk

Kita dapat mendefinisikan target risk:

$$R_i^{stat} = w_i \sigma_i.$$

Kemudian:

$$R_i^{stat} \leq R_{i,max}^{stat}.$$

Ini bisa menjadi constraint tambahan.

N.31. Tetapi kita harus hati-hati dengan double counting

Jika:

$$\Sigma$$

sudah masuk objective:

$$\gamma w^T \Sigma w,$$

kita tidak perlu memberikan penalty volatility individual terlalu besar lagi.

Kalau tidak:

$$\sigma_i$$

akan dihukum dua kali.

Karena itu:

SL risk digunakan sebagai hard/soft constraint, sedangkan covariance digunakan sebagai portfolio risk objective.

Ini desain yang lebih bersih.

N.32. Long dan short

Sekarang sistem kita harus mampu:

$$x_i^{long}$$

dan:

$$x_i^{short}.$$

Kita tidak bisa menggunakan satu binary variable lagi.

Gunakan:

$$x_i^L \in \{0, 1\},$$

$$x_i^S \in \{0, 1\}.$$

N.33. Mutually exclusive long-short

Tidak boleh:

$$x_i^L = x_i^S = 1.$$

Constraint:

$$x_i^L + x_i^S \leq 1.$$

Penalty:

$$P_{LS} x_i^L x_i^S.$$

N.34. Signed position

Position:

$$w_i = w_i^L - w_i^S.$$

Dengan:

$$w_i^L = \sum_k q_{i,k}^L P_i x_{i,k}^L$$

dan:

$$w_i^S = \sum_k q_{i,k}^S P_i x_{i,k}^S.$$

Maka:

$$w_i = w_i^L - w_i^S.$$

N.35. Expected return short

Jika:

$$\hat{\mu}_i < 0,$$

short dapat memiliki expected profit:

$$E[P nL_i^{short}] = -w_i \hat{\mu}_i.$$

Secara umum:

$$E[P nL] = \sum_i w_i \hat{\mu}_i.$$

Karena $w_i < 0$ untuk short.

N.36. Covariance untuk long-short

Tidak perlu membuat formula baru.

Tetap:

$$\text{Var}(R_p) = w^T \Sigma w.$$

Karena w_i sekarang signed.

Menariknya, posisi short secara alami menghasilkan perubahan tanda pada covariance term.

N.37. Hedging muncul otomatis

Misalkan:

$$\text{Corr}(A, B) = 0.9.$$

Jika kita long A dan short B:

$$w_A > 0,$$

$$w_B < 0.$$

Maka:

$$w_A w_B \Sigma_{AB} < 0.$$

Portfolio variance dapat turun.

Jadi optimizer dapat menemukan:

$$\text{hedging structure}$$

secara matematis.

N.38. Take Profit

Sekarang kita masuk ke TP.

Misalkan entry:

$$P_i.$$

TP:

$$TP_i.$$

Untuk long:

$$g_i = TP_i - P_i.$$

Reward:

$$G_i = q_i g_i.$$

Stop loss:

$$L_i = q_i d_i.$$

Reward-risk ratio:

$$RR_i = \frac{g_i}{d_i}.$$

N.39. TP tidak harus masuk langsung ke objective

Ini keputusan desain penting.

Expected return:

$$\hat{\mu}_i$$

sudah menggambarkan forecast return.

Jika kita sekaligus memasukkan TP sebagai expected return tanpa model probabilitas menyentuh TP, kita akan melakukan double counting.

Jadi untuk V1:

$$TP \rightarrow \text{execution/risk layer}$$

bukan langsung:

$$TP \rightarrow \text{expected return.}$$

N.40. Tetapi TP dapat digunakan sebagai feasibility constraint

Misalnya kita hanya mau trade jika:

$$E[R_i]$$

cukup besar dibandingkan stop risk.

Definisikan:

$$RR_i^{\text{expected}} = \frac{\hat{\mu}_i}{d_i/P_i}.$$

Kita dapat meminta:

$$RR_i^{\text{expected}} \geq RR_{\min}.$$

N.41. Expected return relatif terhadap stop

Fractional stop distance:

$$s_i = \frac{d_i}{P_i}.$$

Expected return:

$$\hat{\mu}_i.$$

Maka:

$$ER/SL_i = \frac{\hat{\mu}_i}{s_i}.$$

Jika:

$$\frac{\hat{\mu}_i}{S_i}$$

terlalu kecil, trade mungkin tidak menarik.

N.42. Tetapi jangan menyamakan ini dengan probability of TP

Ini penting secara finansial.

$$\hat{\mu}_i$$

adalah conditional expected return.

Sedangkan:

$$P(TP)$$

adalah hitting probability.

Keduanya berbeda.

Jika nanti kita membangun **Barrier-Hitting Model**, barulah TP dan SL dapat masuk secara probabilistik.

Untuk Tahap N kita belum melakukan itu.

N.43. Liquidity constraint

Position tidak boleh lebih besar daripada liquidity.

Misalnya:

$$ADV_i$$

average daily volume.

Kita dapat menetapkan:

$$|w_i| \leq \eta ADV_i P_i.$$

Dengan:

$$\eta$$

misalnya hanya sebagian kecil dari volume yang dianggap aman.

N.44. Untuk short-term trading, order book lebih baik

Karena sistem kita memang akan menggunakan order book, kita dapat mengganti ADV dengan:

$$D_i^{book}$$

depth yang tersedia di sekitar harga entry.

Maka:

$$q_i \leq \eta D_i^{book}.$$

Ini jauh lebih relevan untuk execution.

N.45. Slippage

Jika depth tipis:

$$Slippage_i(q)$$

meningkat.

Jadi cost:

$$C_i(q) = fee_i + spread_i + slippage_i(q).$$

Karena:

$$slippage_i(q)$$

bergantung pada q , objective dapat menjadi nonlinear.

N.46. Untuk QUBO kita discretize cost

Misalnya setiap position level:

$$q_{i,k}$$

mempunyai:

$$C_{i,k}.$$

Maka:

$$Cost = \sum_{i,k} C_{i,k} x_{i,k}.$$

Dengan demikian cost tetap linear dalam binary variables.

Ini salah satu alasan mengapa discretization sangat berguna.

N.47. QUBO final V1

Sekarang kita dapat menulis:

$$E(x) = E_{return} + E_{risk} + E_{cost} + E_{SL} + E_{cardinality} + E_{exclusive} + E_{liquidity}.$$

N.48. Return term

$$E_{return} = - \sum_{i,k} P_i q_{i,k} \hat{\mu}_i x_{i,k}.$$

N.49. Risk term

$$E_{risk} = \gamma \sum_{i,j} \sum_{k,l} P_i P_j q_{i,k} q_{j,l} \Sigma_{ij} x_{i,k} x_{j,l}.$$

N.50. Cost term

$$E_{cost} = \sum_{i,k} C_{i,k} x_{i,k}.$$

N.51. SL-risk constraint

Define:

$$r_{i,k} = q_{i,k} d_i.$$

Constraint:

$$\sum_{i,k} r_{i,k} x_{i,k} \leq R_{\max}.$$

Dengan slack:

$$\sum_{i,k} r_{i,k} x_{i,k} + s = R_{\max}.$$

Penalty:

$$E_{SL} = \lambda_{SL} \left(\sum_{i,k} r_{i,k} x_{i,k} + s - R_{\max} \right)^2.$$

N.52. Cardinality

Misalnya maksimal:

$$K_{\max}$$

positions.

$$\sum_i y_i + s_K = K_{\max}.$$

Penalty:

$$E_{cardinality} = \lambda_K \left(\sum_i y_i + s_K - K_{\max} \right)^2.$$

N.53. Long-short exclusivity

$$E_{exclusive} = \lambda_{LS} \sum_i x_i^L x_i^S.$$

N.54. Liquidity

Jika position level sudah dibatasi sehingga:

$$q_{i,k} \leq q_{i,max}^{liquidity},$$

maka liquidity constraint sebenarnya dapat diselesaikan **sebelum** QUBO.

Ini lebih baik daripada menambahkan penalty.

Kita dapat menghapus position levels yang tidak feasible.

N.55. Ini adalah prinsip desain penting

Tidak semua constraint harus menjadi QUBO penalty.

Ada tiga jenis:

Hard preprocessing

Contoh:

$$q_{i,k} > q_{max}$$

langsung dibuang.

QUBO penalty

Constraint yang memang perlu dioptimasi bersama.

Objective

Return dan risk.

Dengan demikian QUBO tidak menjadi terlalu besar.

N.56. Final QUBO

Setelah semua ekspansi:

$$E(x) = x^T Q x + C.$$

Kemudian:

$$x^* = \arg \min_{x \in \{0,1\}^M} x^T Q x.$$

Dengan:

$$M = N \times L \times D$$

untuk kombinasi:

- N : assets,
 - L : position levels,
 - D : direction.
-

N.57. Contoh ukuran problem

Misalkan:

$$N = 20$$

aset.

Position levels:

$$L = 3.$$

Direction:

$$D = 2.$$

Maka:

$$M = 20 \times 3 \times 2 = 120$$

binary variables.

QUBO:

$$Q \in \mathbb{R}^{120 \times 120}.$$

Kemudian:

$$120$$

qubits diperlukan untuk direct QAOA encoding.

Ini mulai menjadi problem yang serius.

N.58. Dan di sinilah quantum advantage tidak boleh diasumsikan

Kita **tidak boleh** mengatakan:

QAOA pasti lebih cepat.

Belum tentu.

Untuk ukuran kecil:

$$N = 4, 8, 12$$

classical solver mungkin jauh lebih cepat.

Nilai penelitian kita adalah menguji:

QAOA vs classical optimization

pada **problem yang sama**.

N.59. Exact solver sebagai ground truth

Untuk problem kecil, kita bisa melakukan exhaustive search:

$$2^M$$

kombinasi.

Kemudian:

$$E_{min} = \min_x E(x).$$

Ini menjadi:

$$E_{ground\ truth}.$$

QAOA kemudian dibandingkan terhadap ground truth.

N.60. Approximation ratio

Jika objective kita adalah minimization:

$$r = \frac{E_{QAOA} - E_{min}}{|E_{min}|}.$$

Semakin:

$$r \rightarrow 0,$$

semakin dekat QAOA terhadap optimum.

N.61. Tetapi untuk trading kita membutuhkan metrik kedua

Quantum optimizer mungkin menemukan solusi dengan:

$$E(x)$$

sangat bagus.

Tetapi belum tentu realized PnL bagus.

Karena itu kita membutuhkan dua level evaluasi:

Optimization quality

$$E(x).$$

Financial quality

PnL, Sharpe, MDD, turnover, hit rate, risk adherence.

N.62. Ini menghasilkan dua eksperimen berbeda

Eksperimen A

Apakah QAOA dapat menemukan solusi QUBO yang dekat dengan optimum?

Eksperimen B

Apakah solusi tersebut menghasilkan trading performance yang baik out-of-sample?

Jangan mencampurkan keduanya.

N.63. Dynamic QUBO

Inilah alasan kita menyebut Tahap N **Dynamic**.

Setiap periode:

t

kita mendapatkan:

$\mu_t, \Sigma_t, P_t, SL_t, C_t, Liquidity_t.$

Maka:

$$Q_t = Q(\mu_t, \Sigma_t, P_t, SL_t, C_t, Liquidity_t).$$

Jadi:

$$Q_{t+1} \boxminus Q_t$$

secara umum.

N.64. Pasar mengubah Hamiltonian

Ini menurut saya bagian paling menarik dari keseluruhan desain.

Karena:

$$Q_t$$

berasal dari market state, maka:

$$H_C(t)$$

juga berubah.

Secara konseptual:

$$\boxed{Market \rightarrow Q_t \rightarrow H_C(t)}$$

Jadi kita memiliki **time-dependent optimization landscape**.

N.65. Namun QAOA tidak "memprediksi" perubahan landscape

Forecasting layer:

$$F_t \rightarrow \mu_t, \Sigma_t.$$

Optimization layer:

$$(\mu_t, \Sigma_t) \rightarrow Q_t.$$

Quantum layer:

$$Q_t \rightarrow x_t^*.$$

Ini tiga layer yang harus tetap dipisahkan:

$$Prediction \rightarrow Optimization \rightarrow Execution.$$

N.66. Menghubungkannya ke Ising

Setelah QUBO:

$$x_i \in \{0, 1\},$$

kita gunakan:

$$x_i = \frac{1 - Z_i}{2}.$$

Dengan:

$$Z_i \in \{-1, +1\}.$$

Substitusi:

$$x^T Q x$$

menghasilkan:

$$H_C = \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j + C.$$

Konstanta C tidak memengaruhi argmin.

N.67. Interpretasi fisiknya

Sekarang setiap qubit merepresentasikan keputusan:

$$Z_i = -1$$

atau:

$$Z_i = +1.$$



Yang secara ekonomi berarti:

trade tidak dipilih

atau:

trade dipilih.

Kemudian:

$$J_{ij}$$

merepresentasikan interaction antara dua keputusan.

N.68. Hubungan dengan covariance

Ini sangat menarik.

Karena QUBO memiliki:

$$\Sigma_{ij},$$

maka Ising coupling akan mengandung:

$$J_{ij} \sim \gamma P_i P_j q_i q_j \Sigma_{ij}.$$

Jadi:

correlation structure pasar diterjemahkan menjadi coupling antar-qubit.

Asset yang highly correlated:

$$\Sigma_{ij} > 0$$

menghasilkan coupling tertentu yang membuat pemilihan keduanya menjadi kurang/lebih energetically favorable tergantung konvensi tanda Hamiltonian.

N.69. Factor model bahkan masuk secara tidak langsung ke coupling

Ingat:

$$\Sigma = B \Sigma_F B^T + \Sigma_\epsilon.$$

Maka:

$$J_{ij} \sim \gamma P_i P_j (B_i \Sigma_F B_j^T + \Sigma_{\epsilon,ij}).$$

Artinya:

$$\boxed{J_{ij}}$$

sebenarnya membawa informasi:

- BTC exposure,
- ETH exposure,
- Gold exposure,
- USD exposure,
- equity exposure,
- volatility exposure,
- residual correlation.

Dengan kata lain:

market structure menjadi quantum interaction structure.

Ini adalah salah satu bagian paling kuat dari arsitektur penelitian kita.

N.70. Contoh sederhana

Misalkan terdapat:

BTC, ETH, Gold.

Factor matrix:

$$B = \begin{bmatrix} 1.0 & 0.5 & 0 & 0 \\ 0.8 & 0.7 & 0 & 0 \\ 0 & 0 & 1.2 & -0.3 \end{bmatrix}.$$

Dengan factor:

$$[BTC, ETH, Gold, DXY].$$

Maka covariance asset tidak lagi muncul sebagai angka arbitrer.

Ia berasal dari:

$$B\Sigma_F B^T + \Sigma_\epsilon.$$

N.71. Ini membuat sistem kita jauh lebih interpretable

Kita dapat menjawab:

Mengapa optimizer tidak memilih asset A?

Bisa jadi:

$$\hat{\mu}_A$$

rendah.

Atau:

$$\Sigma_{AA}$$

tinggi.

Atau:

$$\Sigma_{AB}$$

tinggi dengan posisi B yang sudah dipilih.

Atau:

$$SL_A$$

terlalu dekat/jauh.

Atau:

$$cost_A$$

terlalu besar.

Atau risk budget sudah habis.

Ini jauh lebih baik daripada black-box:

$$input \rightarrow BUY.$$

N.72. Position sizing dan SL sekarang terhubung

Kita dapat merumuskan:

$$q_{i,k} = \frac{\rho_k R_{\max}}{d_i},$$

dengan:

$$\rho_k \in (0, 1].$$

Misalnya:

$$\rho = \{0.25, 0.5, 1.0\}.$$

Maka setiap asset secara otomatis memiliki position size yang menyesuaikan SL.

N.73. Contoh

Asset A:

$$d_A = 0.5.$$

Asset B:

$$d_B = 2.$$

Dengan:

$$R_{\max} = 1.$$

Maka maximum position:

$$q_A^{\max} = 2,$$

sedangkan:

$$q_B^{\max} = 0.5.$$

Jadi asset dengan SL lebih lebar secara otomatis mendapatkan position lebih kecil.

N.74. Ini menjawab salah satu masalah utama trading

Kita tidak boleh mengatakan:

setiap trade membeli nominal yang sama.

Karena:

\$100

pada asset volatility rendah tidak sama risikonya dengan:

\$100

pada asset volatility tinggi.

Risk-based sizing memperbaiki hal tersebut.

N.75. Tetapi \$1 bukan berarti guaranteed loss = \$1

Ini harus kita definisikan dengan benar.

Secara model:

$$R_i = q_i d_i \leq 1.$$

Tetapi realized loss dapat lebih besar karena:

slippage,

gap,

market impact,

atau:

execution failure.

Jadi \$1 adalah:

modelled stop risk

bukan jaminan matematis bahwa realized loss selalu tepat \$1.

N.76. Maka kita dapat menambahkan safety multiplier

Misalnya:

$$\kappa > 1.$$

Effective risk budget:

$$R_i^{\text{effective}} = \kappa q_i d_i.$$

Jika:

$$\kappa = 1.2,$$

kita menganggap ada 20% buffer.

Namun nilai κ nantinya harus **diestimasi dari data execution**, bukan dipilih sembarangan.

N.77. Sekarang kita memiliki formulasi V1 yang cukup lengkap

Saya akan merangkum objective utama kita:

$$\begin{aligned}
\min_x \quad & - \sum_{i,k} P_i q_{i,k} \tilde{\mu}_i x_{i,k} \\
& + \gamma \sum_{i,j} \sum_{k,l} P_i P_j q_{i,k} q_{j,l} \sum_{ij} x_{i,k} x_{j,l} \\
& + \sum_{i,k} C_{i,k} x_{i,k} \\
& + \lambda_{SL} \left(\sum_{i,k} r_{i,k} x_{i,k} + s - R_{\max} \right)^2 \\
& + \lambda_K \left(\sum_i y_i + s_K - K_{\max} \right) \\
& + \lambda_{LS} \sum_i x_i^L x_i^S.
\end{aligned}$$

Dengan:

$$r_{i,k} = q_{i,k} d_i.$$

N.78. Ini kemudian menjadi

$$E(x) = x^T Q_t x + C_t.$$

Kemudian:

$$x_t^* = \arg \min_x x^T Q_t x.$$

Dan:

$$x_t^* \rightarrow Trade\ Plan_t.$$

N.79. Trade Plan

Output optimizer bukan langsung:

BUY BTC.

Tetapi sesuatu seperti:

Asset	Direction	Level	Position	SL	TP
BTC	LONG	2	0.004 BTC	...	...
ETH	LONG	1	0.010 ETH	...	...
SOL	NONE	-	0	-	-

Kemudian execution engine baru menentukan:

limit price,

order book placement,

TP,

SL.

N.80. Ini juga menjaga desain kita dari masalah besar

Optimizer **tidak boleh** mengatur execution microstructure secara langsung.

QUBO menentukan:

WHAT

sedangkan execution engine menentukan:

HOW

Contoh:

Optimizer:

Long BTC, risk \$1, position level 2.

Execution engine:

Letakkan limit order pada bid/ask tertentu, tunggu fill, pasang SL/TP, monitor order book.

Pemisahan ini akan sangat penting ketika kita nanti benar-benar menghubungkannya ke Binance dan kemudian broker saham.

N.81. Dynamic re-optimization

Misalkan order belum filled.

Pada:

$$t_0 :$$

kita memiliki:

$$Q_{t_0}.$$

Satu jam kemudian:

$$t_1 :$$

market berubah:

$$Q_{t_1} \not\equiv Q_{t_0}.$$

Maka sistem menghitung ulang:

$$x_{t_1}^*.$$

Jika trade lama tidak lagi optimal, order dapat:

cancel.

Kemudian:

replace.

Ini persis menjawab rancangan awalmu mengenai:

setiap periode mengevaluasi apakah pending order masih layak dipertahankan.

N.82. Untuk posisi yang sudah filled

Kita harus membedakan:

Pending Order

dengan:

Open Position.

Untuk pending:

Q_t

dapat digunakan langsung untuk memutuskan:

keep/cancel/replace.

Untuk open position, kita tidak langsung melakukan liquidation hanya karena:

x_t^*

berubah.

Kita membutuhkan **Position Management Layer**.

Itu akan menjadi bagian dari tahap berikutnya.

N.83. Mengapa?

Misalnya:

$t_0 :$

model berkata:

BUY.

Trade filled.

Kemudian:

$t_1 :$

model berubah menjadi:

HOLD.

Tidak berarti:

SELL

secara otomatis.

Kita harus mempertimbangkan:

- unrealized PnL,
- time-in-trade,
- distance to SL,
- distance to TP,
- updated expected return,
- updated volatility,
- updated factor regime,
- opportunity cost.

Jadi keputusan:

ENTER

dan:

MANAGE

adalah dua masalah optimasi yang berbeda.

N.84. Ini sangat cocok dengan desain awal kita

Kita sekarang mempunyai:

Optimization Problem A : Entry Selection

dan nantinya:

Optimization Problem B : Position Management.

Problem B akan menjawab:

Apakah posisi tetap dipertahankan?

Apakah SL dinaikkan ke BEP?

Apakah TP diperpanjang?

Apakah sebagian posisi ditutup?

N.85. Bahkan itu bisa menjadi QUBO kedua

Misalnya sebuah posisi memiliki:

3

aksi:

$a_1 = \text{KEEP},$

$a_2 = \text{MOVE SL},$

$a_3 = \text{PARTIAL EXIT}.$

Maka kita dapat membangun:

$$x_{i,a} \in \{0, 1\}$$

dan mengoptimasi expected future utility.

Jadi arsitektur akhirnya bukan hanya satu QUBO.

Kita dapat mempunyai:

$QUBO_{entry}$

dan:

$QUBO_{management}.$

Ini akan sangat menarik secara penelitian.

N.86. Posisi kita sekarang

Setelah Tahap N:

$$Market \rightarrow Factor \rightarrow Forecast \rightarrow Risk \rightarrow Position\ Sizing \rightarrow QUBO \rightarrow Ising \rightarrow QAOA$$

sudah tersambung secara matematis.

Dan yang paling penting, setiap layer memiliki fungsi yang jelas:

Layer	Pertanyaan
Data	Apa yang sedang terjadi?
Factor	Faktor apa yang menggerakkan pasar?
Forecast	Apa yang mungkin terjadi berikutnya?
Risk	Seberapa tidak pasti?
Position sizing	Berapa besar posisi yang aman?
QUBO	Kombinasi trade mana yang optimal?
Ising	Bagaimana merepresentasikan optimasi secara quantum?
QAOA	Bagaimana mencari state energi rendah?
Execution	Bagaimana mengeksekusi keputusan?
Management	Apakah posisi perlu dipertahankan/diubah?

Persamaan inti Tahap N

Kalau seluruh Tahap N harus dipadatkan menjadi beberapa persamaan utama, saya akan mempertahankan enam ini:

1. Position sizing

$$q_{i,k} = \frac{\rho_k R_{\max}}{d_i}$$

2. Net expected return

$$\tilde{\mu}_i = \mu_i - C_i$$

3. Portfolio risk

$$\sigma_p^2 = w^T \Sigma w$$

4. Risk budget

$$\sum_i q_i d_i \leq R_{\max}$$

5. Dynamic QUBO

$$Q_t = Q(\mu_t, \Sigma_t, P_t, SL_t, C_t, R_t)$$

6. Optimization

$$x_t^* = \arg \min_{x \in \{0,1\}^M} x^T Q_t x$$

Kemudian:

$$x_i = \frac{1 - Z_i}{2}$$

memberikan:

$$H_C = \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j.$$

Satu konsekuensi penting dari Tahap N

Kita sekarang dapat melihat dengan sangat jelas **di mana matematika quantum benar-benar masuk ke quantitative trading kita.**

Bukan pada forecasting:

$$P_{t+1} = f(P_t),$$

dan bukan pada prediksi BTC.

Quantum masuk setelah seluruh informasi pasar telah direduksi menjadi:

$$\boxed{\mu_t, \Sigma_t, SL_t, C_t, R_t.}$$

Informasi tersebut membentuk **QUBO**, dan QUBO membentuk **Ising Hamiltonian**. Dengan demikian, apabila nanti kita membandingkan **Exact Solver vs Classical Optimizer vs VQE/QAOA**, ketiganya benar-benar menerima **problem matematis yang identik**. Ini sangat penting agar penelitian kita tidak berubah menjadi sekadar "menggunakan quantum karena quantum", tetapi menjadi eksperimen yang dapat dipertanggungjawabkan.

Tahap berikutnya secara logis adalah **Tahap O — Position Management & Dynamic Rebalancing**: kita akan memformalkan bagian yang sejak awal paling kamu inginkan, yaitu **apa yang dilakukan setelah order pending/filled**, termasuk floating loss, floating profit, break-even stop, trailing SL, dynamic TP, partial exit, time stop, dan kapan sistem harus membangun ulang QUBO. Di sana kita bisa mulai membentuk **QUBO kedua untuk pengelolaan posisi yang sudah terbuka**.